

G-Code Audit Engine — Changelog

Version v3 | Date 05/02/26 | File gcode_audit_v3_050226.py

Previous version	v2 (g-code_verification_v1.py, milestone 050126_v2)
This version	v3 (gcode_audit_v3_050226.py)
Scope of pass	Parser expansion + tokenizer rewrite + M03/M3 normalization fix
New validators added	None (deliberate — scoped per agreement)
Breaking changes	None — parser output remains backward-compatible
Test corpus	test.gcode (Autodesk Fusion / OpenBuilds GRBL post, 110 lines)

Summary of What Changed

- **Tokenizer rewrite.** Replaced naive `str.split()` with a regex-based G-code word tokenizer. Now handles space-separated form (G1 X10 Y20), no-space form (G1X10Y20), and mid-line comments correctly.
- **Comment handling.** Both parenthetical (. . .) and semicolon-to-end-of-line ; . . . styles are now stripped before tokenization. Multi-comment lines and inline comments between tokens (X10 (note) Y20) work correctly.
- **Code normalization.** Numeric G/M/T codes are normalized at parse time: G01 → G1, M03 → M3, G00 → G0. This fixes the v2 spindle/feed validator bug where zero-padded codes from real-world post-processors (Fanuc, Haas) silently failed to match.
- **Modal state expansion.** The per-line state dict now carries full modal context: motion mode (G0/G1/G2/G3), distance (G90/G91), units (G20/G21), plane (G17/G18/G19), spindle (M3/M4/M5), coolant (M7/M8/M9), feed (F), speed (S), and tool (T) — in addition to X/Y/Z position.
- **Modal inheritance verified.** Continuation lines without a G-code (e.g. line 57 of test.gcode: Z-1.058) correctly inherit the active motion mode (G1) from the previous line. Verified end-to-end against test.gcode.
- **Spindle/feed validator rewrite.** `validate_spindle_and_feed` now reads modal state instead of re-scanning raw tokens. Cleaner code, and the M03/M3 normalization fix flows through automatically.
- **Line classification.** Each parsed line is now tagged `line_type`: CODE, COMMENT, or EMPTY. Validators can filter to CODE-only lines without re-checking token contents.
- **Backward compatibility preserved.** Parser output shape unchanged: `{"lines": [{"line_index", "state", "tokens", "line_type"}, ...]}`. Existing validators (`validate_startup_sequence`, `group_operations`, `profile_operation_depths`) work unchanged. Same output on test.gcode as v2 — no regressions.

Test Results

Run against `test.gcode` (Autodesk Fusion / OpenBuilds GRBL post-processor output, 110 lines, 2 cutting operations):

Check	v2 result	v3 result	Status
Lines parsed	110	110 (64 code / 39 comment / 7 empty)	✓ match
Sequence issues	(none)	(none)	✓ match
Operations found	2	2	✓ match
Depth profile	-9.5250 (1), -3.3100 (1)	-9.5250 (1), -3.3100 (1)	✓ match
Spindle/feed issues	(none)	(none)	✓ match
Final modal state	n/a	G0, G90, G21, G17, M5, F2800, S18000	✓ new

Edge case spot-checks (run separately):

- No-space tokens: G1X10Y20Z-1.5 → ['G1', 'X10', 'Y20', 'Z-1.5']
- Zero-padded G code: G01 X10.0 Y20.0 → ['G1', 'X10.0', 'Y20.0']
- Zero-padded M code: M03 S18000 → ['M3', 'S18000']
- Inline comment: G1 X10 (rapid here) Y20 → ['G1', 'X10', 'Y20']
- Semicolon comment: G1 X10 ; comment → ['G1', 'X10']

Known Limitations / Deliberately Deferred

- **G91 incremental coordinates not interpreted.** The parser *tracks* distance mode (G90/G91) in state but does not yet *interpret* G91 incremental coordinates. X/Y/Z values are always treated as absolute positions. There is a TODO comment in `_apply_tokens_to_state` flagging this. Programs using G91 will produce inaccurate position state. **Recommended for next pass.**
- **Arc geometry not validated.** G2/G3 arcs are tokenized correctly and motion mode is captured, but the actual arc path (using I/J/K offsets or R radius) is not yet computed or validated. Future audits like "arc with invalid geometry" or "arc clearance check" will need an arc-resolution helper.
- **"Safe Z" defined as Z > 0.** Inherited from v2. This assumes work-zero is at the top of the workpiece, which is common but not universal. Should become machine-config-driven before this tool is used across different setups.
- **Malformed tokens silently ignored.** If a coordinate fails to parse as a float (e.g. X1.2.3), the parser skips it without raising. No issue is currently emitted. Worth adding a low-severity informational issue type in a future pass.
- **Modal groups not exhaustive.** Tracked: motion (group 1), plane (2), distance (3), units (6), spindle, coolant. Not yet tracked: feed-rate mode (G93/G94/G95), cutter compensation (G40/G41/G42), tool length offset (G43/G49), work coordinate system (G54-G59). These are preserved in the tokens list but don't update persistent state. Add as needed when a validator requires them.

Suggestions for Next Pass

- **G91 incremental coordinate handling.** Update `_apply_tokens_to_state` so that when `state["distance"] == "G91"`, X/Y/Z values are added to the previous position rather than replacing it. Need to decide how to handle the first G91 motion when no prior position exists.

- **Begin module split.** Per the agreed structure: extract parser + tokenizer + state model into `gcode_audit_core_v4.py`; move existing validators into per-concern files (`gcode_audit_startup_v4.py`, `gcode_audit_spindle_v4.py`, etc.); add a thin runner. Do this *before* adding new validators so they go into the right place from the start.
- **First new validator candidate: rapid-into-material.** Detect G0 motions that occur with Z below clearance, or G0 moves that change XY while Z is below clearance. This is the highest-value safety audit not yet implemented and is now possible because v3 tracks motion mode. Listed as Tier 1 in the audit catalog.
- **Configuration scaffolding.** Many planned audits need parameters (safe Z height, machine envelope, max plunge ratio). Even a minimal `config.py` with named constants would be a good start, before parameters get hardcoded across multiple validators.

This changelog accompanies `gcode_audit_v3_050226.py`. A matching changelog PDF will be produced for each future code package.